

Unit 3. Basic Concepts of strings and Exceptions**3.1 Strings**

3.1.1 Basic string operations, String comparison

3.1.2 String methods (charAt(), concat(). equals(), indexOf(), isEmpty(), join(),lastIndexOf(),length(),split(), substring().trim())

3.1.3 StringBuffer class and its constructors.

3.1.4 StringBuffer methods : (append(),insert(),update(), delete(). reverse(),capacity())

3.2 Introduction to Exceptions:

3.2.1 Exception Types, User defined Exception

3.2.2 Throw,Throws

3.2.3 Try, CatchandFinally

3.1 Strings

String is a sequence of characters. In java, objects of String are immutable which means it is a constant and cannot be changed once created.

Creating a String

There are two ways to create string in Java:

String literal

String s = "LetsLearnJava";

Using *new* keyword

String s = new String ("LetsLearnJava");

Immutable String in Java

A String that can be modified or changed is known as mutable String. StringBuffer and StringBuilder classes are used for creating mutable strings.

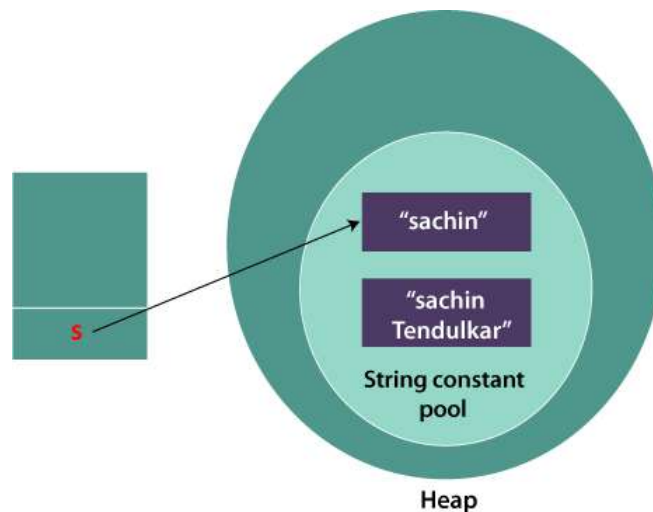
A String is an unavoidable type of variable while writing any application program. String references are used to store various attributes like username, password, etc. In Java, **String objects are immutable**. Immutable simply means unmodifiable or unchangeable.

Once String object is created its data or state can't be changed but a new String object is created.

```
class Testimmutablestring{
    public static void main(String args[]){
        String s="Sachin";
        s.concat(" Tendulkar");//concat() method appends the string at the end
        System.out.println(s);//will print Sachin because strings are immutable objects
    }
}
```

Output:

Sachin



two objects are created but `s` reference variable still refers to "Sachin" not to "Sachin Tendulkar".

But if we explicitly assign it to the reference variable, it will refer to "Sachin Tendulkar" object.

```
class Testimmutablestring1{  
    public static void main(String args[]){  
        String s="Sachin";  
        s=s.concat(" Tendulkar");  
        System.out.println(s);  
    }  
}
```

Output:

Sachin Tendulkar

3.1.1 String Comparison

We can compare String in Java on the basis of content and reference.

It is used in **authentication** (by `equals()` method), **sorting** (by `compareTo()` method), **reference matching** (by `==` operator) etc.

There are three ways to compare String in Java:

1. By Using equals() Method
2. By Using == Operator
3. By compareTo() Method

1) By Using equals() Method

The String class equals() method compares the original content of the string. It compares values of string for equality. String class provides the following two methods:

- **public boolean equals(Object another)** compares this string to the specified object.
- **public boolean equalsIgnoreCase(String another)** compares this string to another string, ignoring case.

```
class Teststringcomparison1{
    public static void main(String args[]){
        String s1="Java";
        String s2="Java";
        String s3=new String("Java");
        String s4="Language";
        System.out.println(s1.equals(s2));//true
        System.out.println(s1.equals(s3));//true
        System.out.println(s1.equals(s4));//false
    }
}
```

In the above code, two strings are compared using **equals()** method of **String** class. And the result is printed as boolean values, **true** or **false**.

```
class Teststringcomparison2{
    public static void main(String args[]){
        String s1="Java";
        String s2="JAVA";

        System.out.println(s1.equals(s2));//false
        System.out.println(s1.equalsIgnoreCase(s2));//true
    }
}
```

2) By Using == operator

The == operator compares references not values.

```
class Teststringcomparison3{
    public static void main(String args[]){
        String s1="Java";
        String s2="Java";
        String s3=new String("Java");
        System.out.println(s1==s2);//true (because both refer to same instance)
        System.out.println(s1==s3);//false(because s3 refers to instance created in nonpool)
    }
}
```

3) By Using compareTo() method

The String class compareTo() method compares values lexicographically and returns an integer value that describes if first string is less than, equal to or greater than second string.

Suppose s1 and s2 are two String objects. If:

- **s1 == s2** : The method returns 0.
- **s1 > s2** : The method returns a positive value.
- **s1 < s2** : The method returns a negative value.

```
class Teststringcomparison4{
    public static void main(String args[]){
        String s1="Java";
        String s2="Java";
        String s3="Language";
        System.out.println(s1.compareTo(s2));//0
        System.out.println(s1.compareTo(s3));//1(because s1>s3)
        System.out.println(s3.compareTo(s1));//-1(because s3 < s1 )
    }
}
```

3.1.2 String Methods

Method	Description
charAt(int index)	Returns the character at the specified index.
compareTo(String anotherString)	Compares two strings lexicographically.
concat(String str)	Concatenates the specified string to the end of this string.
equals(Object anObject)	Compares this string to the specified object.
equalsIgnoreCase(String anotherString)	Compares this string to another string, ignoring case considerations.
indexOf(int ch)	Returns the index within this string of the first occurrence of the specified character.
indexOf(String str)	Returns the index within this string of the first occurrence of the specified substring.
isEmpty()	Returns true if, and only if, length() is 0.
length()	Returns the length of this string.
replace(char oldChar, char newChar)	Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.
split(String regex)	Splits this string around matches of the given regular expression.
substring(int beginIndex)	Returns a new string that is a substring of this string.
toLowerCase()	Converts all of the characters in this String to lower case.
toUpperCase()	Converts all of the characters in this String to upper case.
trim()	Returns a string whose value is this string, with any leading and trailing whitespace removed.

```

import java.io.*;
import java.util.*;
class StringMethodsDemo
{
    public static void main (String[] args)
    {
        String s= "LetsLearnJava";
    }
}

```

```
// or String s= new String ("LetsLearnJava");
System.out.println("String= " + s);

// Returns the number of characters in the String.
System.out.println("String length = " + s.length());

// Returns the character at ith index.
System.out.println("Character at 3rd position = " + s.charAt(3));

// Return the substring from the ith index character to end of string
System.out.println("Substring " + s.substring(3));

// Returns the substring from i to j-1 index.
System.out.println("Substring = " + s.substring(2,5));

// Concatenates string2 to the end of string1.
String s1 = "Java";
String s2 = "LetsLearn";
System.out.println("Concatenated string = " + s1.concat(s2));

// Returns the index within the string
// of the first occurrence of the specified string.
String s4 = "Learn Share Learn";
System.out.println("Index of Share " + s4.indexOf("Share"));

// Returns the index within the string of the
// first occurrence of the specified string,
// starting at the specified index.
System.out.println("Index of a = " + s4.indexOf('a',3));

// Checking equality of Strings
Boolean out = "Java".equals("java");
System.out.println("Checking Equality " + out);
out = "Java".equals("Java");
System.out.println("Checking Equality " + out);
out = "Java".equalsIgnoreCase("jAvA ");
System.out.println("Checking Equality " + out);

//If ASCII difference is zero then the two strings are similar
```

```

        int out1 = s1.compareTo(s2);
        System.out.println("the difference between ASCII value is="+out1);

        // Converting cases
        String word1 = "LeaRnJava";
        System.out.println("Changing to lower Case " + word1.toLowerCase());

        // Converting cases
        String word2 = "HeLLoHowRu";
        System.out.println("Changing to UPPER Case " + word2.toUpperCase());

        // Trimming the word
        String word4 = "  Learn Share Learn  ";
        System.out.println("Trim the word " + word4.trim());

        // Replacing characters
        String str1 = "JavaForBeginners";
        System.out.println("Original String " + str1);
        String str2 = "JavaForBeginners".replace('n','O') ;
        System.out.println("Replaced n with O -> " + str2);
    }
}

```

- **Reverse string without using reverse method**

```

public class ReverseString {
    public static String reverse(String input) {
        StringBuilder reversed = new StringBuilder();

        // Iterate through the string from the end to the beginning
        for (int i = input.length() - 1; i >= 0; i--) {
            reversed.append(input.charAt(i));
        }

        return reversed.toString();
    }

    public static void main(String[] args) {
        String input = "Hello World!";
    }
}

```

```
        String result = reverse(input);
        System.out.println("Original: " + input);
        System.out.println("Reversed: " + result);
    }
}
```

3.1.3 StringBuffer class and its constructors.

The StringBuffer class in Java is a mutable sequence of characters. It is part of the java.lang package and is used when a lot of modifications are required on a string (like appending, inserting, deleting, etc.). Unlike String, which is immutable, StringBuffer allows direct modification of the object without creating a new one.

Key Features of StringBuffer:

- **Thread-safe:** All methods are synchronized, making it safe to use in multi-threaded environments.
- **Resizable:** Its capacity grows automatically when needed.
- **Performance:** Slower than StringBuilder due to synchronization.

Constructors in StringBuffer

The StringBuffer class provides several constructors to create and initialize objects:

1. **Default Constructor:**
Syntax: StringBuffer()
Description: Creates an empty StringBuffer with an initial capacity of 16 characters.
2. **Constructor with Initial Capacity:**
Syntax: StringBuffer(int capacity)
Description: Creates an empty StringBuffer with the specified initial capacity.
3. **Constructor with String:**
Syntax: StringBuffer(String str)
Description: Creates a StringBuffer initialized with the given string. The capacity is str.length() + 16.
4. **Constructor with CharSequence:**
Syntax: StringBuffer(CharSequence cs)
Description: Creates a StringBuffer initialized with the specified CharSequence (like a String or StringBuilder).

```
public class StringBufferExample {
    public static void main(String[] args) {
        // Default constructor
        StringBuffer sb1 = new StringBuffer();
    }
}
```



```

System.out.println("Default Capacity: " + sb1.capacity()); // 16

// Constructor with initial capacity
StringBuffer sb2 = new StringBuffer(30);
System.out.println("Initial Capacity: " + sb2.capacity()); // 30

// Constructor with String
StringBuffer sb3 = new StringBuffer("Java");
System.out.println("Content: " + sb3); // Java
System.out.println("Capacity: " + sb3.capacity()); // 20

// Constructor with CharSequence
CharSequence cs = "Programming";
StringBuffer sb4 = new StringBuffer(cs);
System.out.println("Content: " + sb4); // Programming
    }
}

```

3.1.4 StringBuffer methods : (append(),insert(),update(), delete(), reverse(),capacity())

Method	Description	Syntax	Example
append()	Adds data to the end of the StringBuffer.	StringBuffer append(DataType data)	sb.append("Java"); → "Hello Java"
insert()	Inserts data at the specified position.	StringBuffer insert(int offset, DataType data)	sb.insert(5, " World"); → "Hello World"
replace()	Replaces a substring with the given string.	StringBuffer replace(int start, int end, String str)	sb.replace(6, 11, "Java"); → "Hello Java"
delete()	Deletes characters between the specified indices.	StringBuffer delete(int start, int end)	sb.delete(5, 11); → "Hello"
reverse()	Reverses the sequence of characters.	StringBuffer reverse()	sb.reverse(); → "dlroW olleH"

Method	Description	Syntax	Example
capacity()	Returns the current capacity of the StringBuffer.	int capacity()	sb.capacity(); → Default 16
ensureCapacity()	Ensures the minimum capacity is at least the specified value.	void ensureCapacity(int minimumCapacity)	sb.ensureCapacity(50);

```

public class StringBufferMethods {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");

        // Append
        sb.append(" World");
        System.out.println("After append: " + sb); // Hello World

        // Insert
        sb.insert(6, "Java ");
        System.out.println("After insert: " + sb); // Hello Java World

        // Replace (Update)
        sb.replace(6, 10, "C++");
        System.out.println("After replace: " + sb); // Hello C++ World

        // Delete
        sb.delete(6, 10);
        System.out.println("After delete: " + sb); // Hello World

        // Reverse
        sb.reverse();
        System.out.println("After reverse: " + sb); // dlroW olleH

        // Capacity
        System.out.println("Capacity: " + sb.capacity()); // Default 16, increases as needed
    }
}

```

Comparison of String and StringBuffer

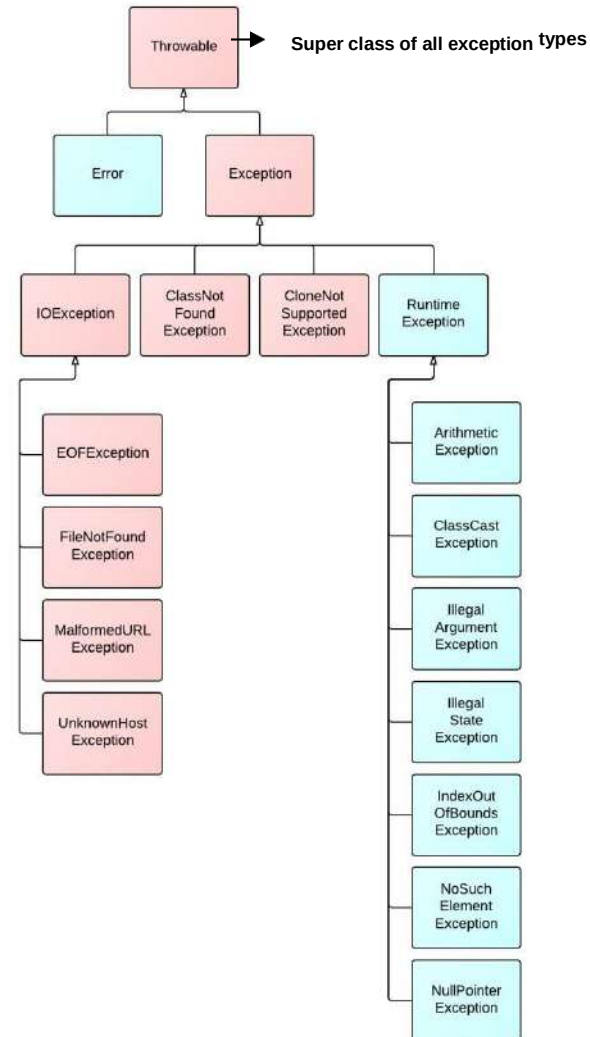
Feature	String	StringBuffer
Mutability	Immutable: Once created, its value cannot be changed.	Mutable: Its content can be modified without creating a new object.
Performance	Slower for frequent modifications as new objects are created for each operation.	Faster for modifications as it operates on the same object.
Thread Safety	Not thread-safe: Requires external synchronization for multi-threaded environments.	Thread-safe: Methods are synchronized, making it safe for use in multi-threaded environments.
Memory Usage	Uses more memory during modifications as new objects are created.	More efficient memory usage as the same object is reused for modifications.
Usage Scenario	Suitable when the string is constant or rarely modified.	Suitable for scenarios where frequent modifications to the string are needed.
String Pool	Supports the string pool for literals to reuse memory.	Does not support the string pool; objects are always created in the heap memory.
Methods Available	concat(), substring(), toUpperCase(), toLowerCase(), trim(), etc.	append(), insert(), replace(), delete(), reverse(), etc.
Synchronization	No built-in synchronization.	Built-in synchronization for thread safety.

3.2 Introduction to Exceptions

- An exception (or exceptional event) is a problem that arises during the execution of a program.

- When an Exception occurs the normal flow of the program is disrupted and the program/Application terminates abnormally, which is not recommended, therefore these exceptions are to be handled.
- Exception can occur at runtime (**known as runtime exceptions**) as well as at compile-time (**known Compile-time exceptions**).

- The '**Throwable**' class is the **superclass** of all errors and exceptions in the Java language and is at the top of exception class hierarchy.
- Subclasses of **throwable** class are **error** and **exception**.
- Errors indicate serious problems and abnormal conditions that most applications should not try to handle. Error defines problems that are not expected to be caught under normal circumstances by our program. For example memory error, hardware error, JVM error etc.
- Exceptions are conditions within the code. A developer can handle such conditions and take necessary corrective actions. Few examples –
 1. **DivideByZero exception**
 2. **NullPointerException**
 3. **ArithmeticException**
 4. **ArrayIndexOutOfBoundsException**
- RuntimeException is a subclass of Exception. Exceptions under this class are automatically defined for programs.



3.2.1 Exception Types

An exception can occur for many different reasons, below given are some scenarios where exception occurs.

- A user has **entered invalid data**.
- A **file** that needs to be opened cannot be found.
- A network connection has been lost in the middle of communications or the JVM has run out of memory.

Some of these exceptions are caused by user error, others by programmer error, and others by physical resources that have failed in some manner. Based on these we have categories exception mainly in **two** types

Checked and Unchecked where **Error** is considered as unchecked exception. The sun microsystem says there are three types of exceptions:

1. **Checked Exception**
2. **Unchecked Exception**
3. **Error**

Checked Exception / Programmatic Exceptions / compile time exceptions:

1. The classes that extend **Throwable** class except **RuntimeException** and **Error** are known as checked exceptions e.g. **IOException**, **SQLException**, File that need to be opened is not found, **ClassNotFoundException**, **IllegalAccessException**, **NoSuchFieldException** **EOFException** etc.
2. The exception that can be predicted by the programmer.
3. Checked exceptions are checked at these are also called as compile time exceptions.
4. Compile-time exceptions cannot simply be ignored at the time of compilation, the Programmer should take care of (handle) these exceptions.

Unchecked Exception / JVM Exceptions / Runtime Exceptions:

- Runtime Exceptions are also known as Unchecked Exceptions as the compiler do not check whether the programmer has handled them or not but it's the duty of the programmer to handle these exceptions and provide a safe exit. In short **Unchecked** exceptions are checked at runtime & ignored at compile time.

- Unchecked exceptions are the class that extends **RuntimeException**. Which means **RuntimeException** is the parent class of all runtime exceptions.
- If we are throwing any runtime exception in a method, it's not required to specify them in the method signature throws clause.
- Runtime Exceptions are caused by bad programming. These include programming bugs, such as logic errors or improper use of an API. For example, if you have declared an array of size 5 in your program, and trying to call the 6th element of the array then an **ArrayIndexOutOfBoundsException** occurs.

Examples

ArithmeticException

ArrayIndexOutOfBoundsException

NullPointerException

NegativeArraySizeException etc.

Error

- **Error is irrecoverable**
- These are not exceptions at all, but problems that arise beyond the control of the user or the programmer.
- Errors are typically ignored in your code because you can rarely do anything about an error. That's why we have a separate hierarchy of errors and we should not try to handle these situations.
- Some of the common Errors are **OutOfMemoryError** and **StackOverflowError**, hardware failure, JVM crash or out of memory error, **VirtualMachineError**, **AssertionError** etc.

Java provides a robust and object oriented way to handle exception scenarios, known as **Java Exception Handling**.

3.2.3, 3.2.2 Try, Catch and Finally, Throw, Throws

Exception handlers

In java, exception handling is done using five keywords,

1. **try**

2. **catch**
3. **throw**
4. **throws**
5. **finally**

When exception occurs, the execution of a program is transfer to an appropriate exception handler

5.3.1 Try & catch:

- The codes which have possibility to generate exceptions are placed in the try block, when an exception occurs, that exception is handled by catch block associated with it. Every try block should be immediately followed either by a catch block or finally block.
- try is the start of the block and catch is at the end of try block to handle the exceptions.
- catch block requires a parameter that should be of type Exception.
- Code within a try/catch block is referred to as protected code,

Syntax:

```
try
{
    //Protected code
}catch(ExceptionName e1)
{
    //Catch block
}
```

Program:

```
class Example1 {
    public static void main(String args[]) {
        int num1, num2;
        try {
            // Try block to handle code that may cause exception
            num1 = 0;
            num2 = 62 / num1;
```

```
        System.out.println("Try block message");
    } catch (ArithmeticException e) {
        // This block is to catch divide-by-zero error
        System.out.println("Error: Don't divide a number by zero");
    }
    System.out.println("I'm out of try-catch block in Java.");
}
```

Output:

```
Error: Don't divide a number by zero
I'm out of try-catch block in Java.
```

5.3.1.1. Multiple catch blocks in Java

We can have multiple catch blocks with a try and try-catch block

Syntax:

try

```
{
    //Protected code
} catch (ExceptionType1 e1)
{
    //Catch block
} catch (ExceptionType2 e2)
{
    //Catch block
} catch (ExceptionType3 e3)
{
    //Catch block
}
```

1. A try block can have any number of catch blocks.
2. A catch block that is written for catching the class Exception can catch all other exceptions

Syntax:

```
catch(Exception e){  
    //This catch block catches all the exceptions  
}
```

3. If multiple catch blocks are present in a program then the above mentioned catch block should be placed at the last as per the exception handling best practices.
4. If the try block is not throwing any exception, the catch block will be completely ignored and the program continues.
5. If the try block throws an exception, the appropriate catch block (if one exists) will catch it
–**catch(ArithmeticException e)** is a catch block that can catch **ArithmeticException**
–**catch(NullPointerException e)** is a catch block that can catch **NullPointerException**
6. All the statements in the catch block will be executed and then the program continues.

Program:

```
class Example2{  
    public static void main(String args[]){  
        try{  
            int a[]=new int[7];  
            a[4]=30/0;  
            System.out.println("First print statement in try block");  
        }  
        catch(ArithmeticException e){  
            System.out.println("Warning: ArithmeticException");  
        }  
        catch(ArrayIndexOutOfBoundsException e){  
            System.out.println("Warning: ArrayIndexOutOfBoundsException");  
        }  
        catch(Exception e){
```

```
        System.out.println("Warning: Some Other exception");
    }
    System.out.println("Out of try-catch block...");
}
}
```

Output:

Warning: ArithmeticException

Out of try-catch block...

In the above example there are multiple catch blocks and these catch blocks executes sequentially when an exception occurs in try block. Which means if you put the last catch block (catch(Exception e)) at the first place, just after try block then in case of any exception this block will execute as it has the ability to handle all exceptions. This catch block should be placed at the last to avoid such situations.

5.3.1.2 Nested try catch blocks in Java

The try catch blocks can be nested. One try-catch block can be present in the another try's body. This is called Nesting of try catch blocks. Each time a try block does not have a catch handler for a particular exception, the stack is unwound and the next try block's catch (i.e., parent try block's catch) handlers are inspected for a match.

If no catch block matches, then the java run-time system will handle the exception.

Syntax:

try

```
{
statement 1;
statement 2;
```

try

```
{
statement 3;
statement 4;
}
```

```
catch(Exception e)
{
//Exception Message
}
} //main try over
catch(Exception e) //Catch of Main(parent) try block
{
//Exception Message
} //main catch over
```

The main point to note here is that whenever the child try-catch blocks are not handling any exception, the control comes back to the parent try-catch if the exception is not handled there also then the program will terminate abruptly.

Program:

```
class Nest{
    public static void main(String args[]){
try{ //Parent try block
    try{ //Child try block1
        System.out.println("Inside block1");
        int b =45/0;
        System.out.println(b);
    }
    catch(ArithmeticException e1){
        System.out.println("Exception: e1");
    }
    //Child try block2
    try{
        System.out.println("Inside block2");
        int b =45/0;
```

```
        System.out.println(b);
    }
    catch(ArrayIndexOutOfBoundsException e2){
        System.out.println("Exception: e2");
    }
    System.out.println("Just other statement");
}
catch(ArithmeticException e3){
    System.out.println("Arithmetic Exception");
    System.out.println("Inside parent try catch block");
}
catch(ArrayIndexOutOfBoundsException e4){
    System.out.println("ArrayIndexOutOfBoundsException");
    System.out.println("Inside parent try catch block");
}
catch(Exception e5){
    System.out.println("Exception");
    System.out.println("Inside parent try catch block");
}
    System.out.println("Next statement..");
}
}
```

Output:

Inside block1

Exception: e1

Inside block2

Arithmetic Exception

Inside parent try catch block

Next statement..

Throw:

throw keyword is used to throw an exception explicitly. Only object of Throwable class or its sub classes can be thrown. Program execution stops on encountering throw statement, and the closest catch statement is checked for matching type of exception. If it doesn't find match it will go for another catch & so..on. if it doesn't find match at all then a default handler halts the program and gives stack trace message. We can throw either checked or unchecked exceptions in java by throw keyword. The throw keyword is mainly used to throw custom exception or user defined exception.

Syntax :

throw **Throwable**object

Creating object of **Throwable** class

There are two possible ways to get an instance of class **Throwable**,

- Using a parameter in catch block.
- Creating instance with new operator.

Example:

throw new IOException("sorry device error");

This constructs an instance of IOException with name sorry device error.

new NullPointerException("test");

This constructs an instance of NullPointerException with name test.

Program:

```
class Test
{
    static void avg()
    {
        try
        {
            int i= 5/0 throw new ArithmeticException("demo");
        }
        catch(ArithmeticException e)
```

```
{  
    System.out.println("Exception caught");  
}  
}
```

```
public static void main(String args[])
```

```
{  
    avg();  
}  
}
```

In the above example the avg() method throw an instance of ArithmeticException, which is successfully handled using the catch statement.

Throws:

Any method capable of causing exceptions must list all the exceptions possible during its execution, so that caller can guard themselves from these exceptions using exception handling codes so normal flow can be maintained. A method can do so by using the throws keyword. Syntax:

type method_name(parameter_list) throws exception_list

```
{  
    //definition of method  
}
```

NOTE : It is necessary for all exceptions, except the exceptions of type Error and RuntimeException, or any of their subclass.

Program:

```
class Test  
{  
    static void check() throws ArithmeticException  
    {  
        System.out.println("Inside check function");
```

```
        throw new ArithmeticException("demo");
    }
    public static void main(String args[])
    {
        try
        {
            check();
        }
        catch(ArithmeticException e)
        {
            System.out.println("caught" + e);
        }
    }
}
```

Advantage of throws:

Now Checked Exception can be propagated (forwarded in call stack).

It provides information to the caller of the method about the exception.

5.3.4 Difference between throw and throws in Java

	Throw	Throws
1.	Throw is statement. It is used to throw an Exception Explicitly.	Throws is a clause. Generally useful for checked exception or for user defined exception used in a method declaration
2.	throw new UnsupportedOperationException("I told you before, no, you can't do that.");	public void doSomethingIllegal() throws UnsupportedOperationException {}
3.	Whenever we want to force an exception then we use throw keyword. the throw keyword (note the singular form) is used to force an exception. It can also pass a custom message to your exception handling module.	the exception can be declared to the method header using the throws keyword followed by the class name of the exception. You might have come across the throws IOException clause in the method header.
4.	The keyword throw is used to throw user defined exceptions and it requires a single argument(a throwable class object) ex: throw new XYZException("Test");	The keyword throws is used in method signatures to declare that, this method could possibly throw an exception. ex: public void test() throws SQLException { }
5.	// This class is of type Throwable class MyClass extends Throwable { // The main method is declared to throw a MyClass Throwable object public static void main(String[] args) throws MyClass { // And let's throw a new instance of MyClass to see what happens. throw new MyClass();	class MyClass { public static void main(String[] args) throws IOException { // body of the method } }

	<pre> } } </pre>	
--	------------------------------	--

1)Java throw example

```

void m(){
    throw new ArithmeticException("sorry");
}

```

2)Java throws example

```

void m()throws ArithmeticException{
    //method code
}

```

3) Java throw and throws example

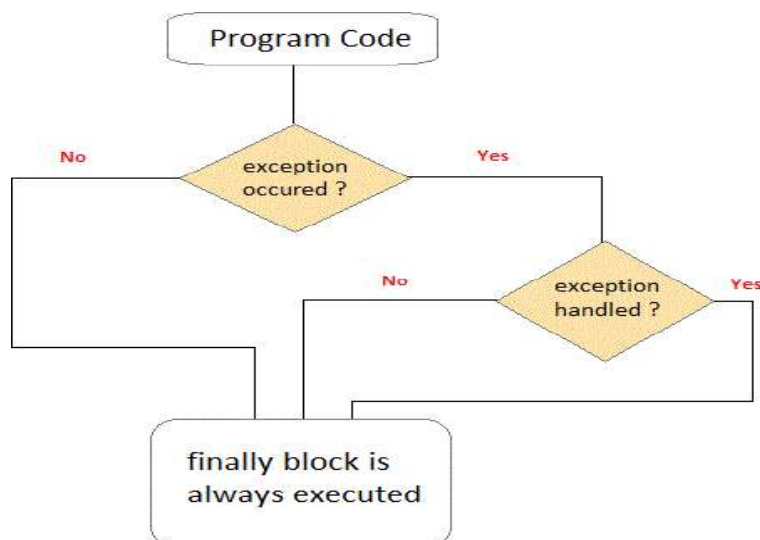
```

void m()throws ArithmeticException{
    throw new ArithmeticException("sorry");
}

```

5.3.5 Finally:

A finally keyword is used to create a block of code that follows a try block. A finally block of code always executes whether or not exception has occurred. A finally block appears at the end of catch block. We can have only one finally block for each try statement.



1. If exception occurs in try block's body then control immediately transferred (**skipping rest of the statements in try block**) to the catch block. Once catch block finished execution then **finally block** completes its execution and then the rest of the program.
2. It is not mandatory to include a finally **block** at all, but if you do, it will run regardless of whether an exception was thrown and handled by the try and catch blocks.
3. If there is no exception occurred in the code which is present in try block then first, the try block gets executed completely and then control gets transferred to finally block (**skipping catch blocks**).
4. If a **return statement** is encountered either in try or catch block. In such case also **finally runs**. Control first goes to finally and then it returned back to **return statement**.

Syntax:

```
try
{
    //Protected code
}

catch(ExceptionType e)
{
    //Catch block
}
: ....
finally
{
    //The finally block always executes.
}
```

Program:

```
class ExceptionTest
{
    public static void main(String[] args)
    {
        int a[]= new int[2];
        System.out.println("out of try");
        try
```

```
{
    System.out.println("Access invalid element"+ a[3]);
    /* the above statement will throw ArrayIndexOutOfBoundsException */
}
finally
{
    System.out.println("finally is always executed.");
}
}
```

Output:

Out of try

finally is always executed.

Exception in thread main java. Lang. exception array Index out of bound exception.

In above example even if exception is thrown by the program, which is not handled by catch block, still finally block will get executed.

3.2.1 User defined Exception

You can create your own exceptions in Java by creating your own exception sub class simply by extending java Exception class. You can define a constructor for your Exception sub class (not compulsory) and you can override the toString() function to display your customized message on catch. **User defined exceptions in java** are also known as **Custom exceptions**.

All exceptions must be a child of Throwable.

If you want to write a checked exception that is automatically enforced by the Handle or Declare Rule, you need to extend the Exception class.

If you want to write a runtime exception, you need to extend the RuntimeException class.

Program1:

```
class MyException extends Exception{
    String str1;
    MyException(String str2) {
        str1=str2;
    }
    public String toString(){
        return ("Output String = "+str1) ;
    }
}
```

```
}  
class CustomException{  
    public static void main(String args[]){  
        try{  
            throw new MyException("Custom");  
            // I'm throwing user defined custom exception above  
        }  
        catch(MyException exp){  
            System.out.println("Hi this is my catch block");  
            System.out.println(exp);  
        }  
    }  
}
```

Output:

Hi this is my catch block

Output String = Custom

Points to Remember

-
- Extend the Exception class to create your own exception class.
 - You don't have to implement anything inside it, no methods are required.
 - You can have a Constructor if you want.
 - You can override the toString() function, to display customized message.
-

Program2:

```
class MyOwnExceptionClass extends Exception {  
    private int price;  
    public MyOwnExceptionClass(int price)  
    {  
        this.price = price;  
    }  
    public String toString()  
    {  
        return "Price should not be in negative, you are entered" +price;  
    }  
}  
  
public class Client {
```

```

public static void main(String[] args)throws Exception
{
    int price = -120;

    if(price < 0)
        throw new MyOwnExceptionClass(price);
    else
        System.out.println("Your price is :"+price);
    }
}

```

Program3:

```

class InvalidAgeException extends Exception{
    InvalidAgeException(String s){
        super(s);
    }
}

class TestCustomException1{
    static void validate(int age)throws InvalidAgeException{
        if(age<18)
            throw new InvalidAgeException("not valid");
        else
            System.out.println("welcome to vote");
    }

    public static void main(String args[]){
        try{
            validate(13);
        }catch(Exception m){System.out.println("Exception occurred: "+m);}

        System.out.println("rest of the code...");
    }
}

```

Rules:

1. We can't have catch or finally clause without a try statement.
2. A try statement should have either catch block or finally block, it can have both blocks.
3. We can't write any code between try-catch-finally block.

4. We can have multiple catch blocks with a single try statement.
 5. Try-catch blocks can be nested similar to if-else statements.
 6. We can have only one finally block with a try-catch statement.
 7. If you do not explicitly use the try catch blocks in your program, java will provide a default exception handler, which will print the exception details on the terminal, whenever exception occurs.
 8. Super class **Throwable** overrides **toString()** function, to display error message in form of string.
 9. While using multiple catch block, always make sure that exception subclasses comes before any of their super classes. Else you will get compile time error.
 10. In nested try catch, the inner try block, uses its own catch block as well as catch block of the outer try, if required.
 11. Only the object of Throwable class or its subclasses can be thrown.
-